

מעבדה 04 — גיבוי Linux · rsync, tar, cron

VM של Ubuntu Server, גיבוי לוקלי + רחוק (SSH), שחזור, אוטומציה ב-cron

רמת קושי: ★★ בינוני

זמן משוער: 60-90 דקות (כולל הקמת VM)

מודול 5 — ניהול מערכות ושירותים

[DR Documentation](#): מעבדה 05 ← הבא

→ הקודם: [Backup VSS](#)

מטרות הלמידה

- להקים VM של Ubuntu Server (התקנה מינימלית מ-ISO)
- להבין את ההבדל בין tar (archive) לבין rsync (sync delta)
- לבצע גיבוי מקומי + מרחוק (rsync over SSH)
- להגדיר SSH key-based authentication לאוטומציה
- לתזמן גיבוי יומי דרך cron
- לבצע שחזור — קובץ ספציפי + תיקייה שלמה
- להשוות לגישה של Windows Server Backup (מהמעבדה הקודמת)

למה המעבדה הזו חשובה

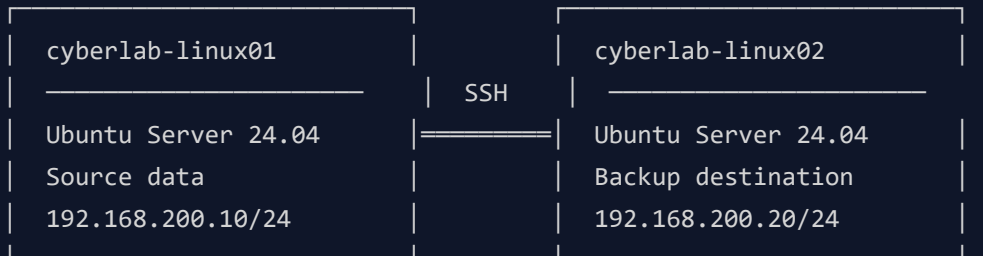
בארגון אמיתי 60-70% מהשרתים = Linux. שרתי web, databases, application servers, קונטיינרים — כולם רצים על **rsync**. Linux הוא הכלי הנפוץ בעולם לגיבוי קבצים — חינמי, מהיר (מעביר רק delta), עובד מקומית ומרחוק דרך SSH. גם אם הקריירה שלכם היא Windows-only, רוב הסבירות שתפגשו שרת Linux איפשהו. הבנת $rsync + cron =$ בסיס הכרחי.

דרישות מקדימות

- ✓ [מדריך VirtualBox](#) הושלם
- ✓ ISO של Ubuntu Server LTS (22.04 או 24.04) — מורידים מ-ubuntu.com/download/server
- זמן: 30-45 דקות נוספות אם זו ההתקנה הראשונה של Ubuntu

⚠ **שימו לב:** המעבדה הזו לא דורשת AD מהמעבדות הקודמות. ה-VMs של Linux רצים בנפרד.

טופולוגיה



(גיבוי מרוחק) `rsync over SSH` כדי להדגים VMs שני *
 .אחד עם 2 דיסקים (גיבוי מקומי בלבד) VM אופציה פשוטה יותר: רק

חלק א' — יצירת VM של Ubuntu (15-30 דקות)

מבוסס על [מדריך VirtualBox](#). צעדים מקוצרים כאן.

שלב 1 — צרו VM חדש

1

:VirtualBox Manager → New

- Name: `cyberlab-linux01`
- ISO: בחרו את Ubuntu Server ISO
- ✓ Skip Unattended
- CPU | RAM: `2048 MB` :RAM: `2`
- Disk: `20 GB`
- Network: Internal Network `linux-net` (חלק ג' של מדריך VirtualBox)

2 שלב 2 — הריצו את ה-VM והתקינו Ubuntu

2

Boot → Start → התקנת Ubuntu Server (תהליך ~15-25 דקות):

- שפה: English
- Keyboard: Default
- Type of install: **Ubuntu Server** (לא Minimized)
- Network: השאירו את ברירת המחדל לעת עתה (נשנה אחרי ההתקנה)
- Storage: **Use entire disk** → Done → Confirm
- Profile:
- Your name: `Lab Admin`
- Server name: `cyberlab-linux01`
- Username: `labadmin`
- Password: `P@ssw0rd-Lab-2026!`
- SSH: סמנו **Install OpenSSH server** ✓ (חובה לאוטומציה אחר כך)
- Snaps: דלגו על הכל
- חכו להתקנה → Reboot

✓ **תוצאה צפויה:** המסך מציג שורת login. הקלידו `labadmin` + הסיסמה. אתם בתוך `bash`.

💡 **טיפ:** קחו snapshot עכשיו: `Clean-Ubuntu-Install`.

שלב 3 — קבעו IP סטטי

3

Ubuntu 24 + משתמש ב-Netplan. ערכו את הקובץ:

```
sudo nano /etc/netplan/50-cloud-init.yaml
```

CLI

החליפו את התוכן ל:

```
network:
  version: 2
  ethernets:
    enp0s3:
      addresses:
        - 192.168.200.10/24
      dhcp4: false
```

CLI

Ctrl+O → Enter → Ctrl+X. החילו:

```
sudo netplan apply
ip a
```

CLI

✓ **תוצאה צפויה:** enp0s3 מציג inet 192.168.200.10/24

⚠ **שימו לב:** אם שם המחשב שלכם שונה (לא enp0s3) — בדקו עם `ip a` ועדכנו את ה-yaml בהתאם. חבילות yml דורשות הזחה של 2 רווחים בדיוק.

שלב 4 — שכפלו את ה-VM ל-cyberlab-linux02

4

כבו את ה-VirtualBox Manager. (`sudo poweroff`) VM → לחצו ימני על

...Clone → cyberlab-linux01

○ Name: cyberlab-linux02

○ MAC Address Policy: **Generate new MAC addresses**

Type: **Full clone** ◦

Clone → לוקח 2-5 דקות.

שלב 5 — הגדירו את linux02

5

הפעילו את `cyberlab-linux02`. היכנסו. שנו IP + hostname:

```
sudo hostnamectl set-hostname cyberlab-linux02
sudo nano /etc/netplan/50-cloud-init.yaml
```

CLI

שנו את ה-IP ל- `192.168.200.20/24`. → Save. `sudo netplan apply`.✓ **תוצאה צפויה:** בדיקת תקשורת — הריצו ב-linux02:

```
ping -c 3 192.168.200.10
```

CLI

✓ **תוצאה צפויה:** 3 חבילות חוזרות. שתי המכונות מתקשרות.**חלק ב' — גיבוי מקומי עם tar (5 דקות)**tar = הכלי הקלאסי של Unix לארכיוב. *Tape ARchive* — מקור השם מימי הסרטים המגנטיים.**שלב 6 — צרו תוכן לגיבוי על linux01**

1

```
mkdir -p ~/business-data
echo "Q1 contract draft" > ~/business-data/contract.txt
echo "Name,Role" > ~/business-data/employees.csv
echo "David,CTO" >> ~/business-data/employees.csv
```

CLI

```
echo "Maya,Engineer" >> ~/business-data/employees.csv  
ls ~/business-data
```

✓ **תוצאה צפויה:** 2 קבצים נוצרו.

שלב 7 — גיבוי מלא עם tar

2

```
sudo mkdir -p /var/backups/local  
sudo tar -czvf /var/backups/local/business-$(date +%Y%m%d).tar.gz ~/bus  
ls -lh /var/backups/local/
```

✓ **תוצאה צפויה:** קובץ `business-20260429.tar.gz` נוצר.

💡 **טיפ:** דגלים: `-c` = `-f` = `-v` = verbose, `-z` = gzip compress, `-z` = create, filename. תזכרו "czvf" — ההגייה: "see-zee-vee-eff".

שלב 8 — שחזור מ-tar

3

מחקו את התיקייה והשחזרו:

```
rm -rf ~/business-data  
tar -xzvf /var/backups/local/business-*.tar.gz -C ~  
ls ~/business-data
```

✓ **תוצאה צפויה:** 2 הקבצים חזרו. `-x` = `.extract`

חלק ג' — rsync מקומי (5 דקות)

rsync = sync חכם. במקום ארכיוב מלא, מעתיק רק שינויים. הרבה יותר מהיר משחזורים שניים.

שלב 9 — בדקו ש-rsync מותקן

1

```
which rsync
rsync --version
```

CLI

✅ **תוצאה צפויה:** מציג נתיב + גרסה. בא מותקן ב-Ubuntu Server.

שלב 10 — sync מ-source ל-destination

2

```
sudo mkdir -p /var/backups/rsync-local
sudo rsync -avh --delete ~/business-data/ /var/backups/rsync-local/
```

CLI

✅ **תוצאה צפויה:** פלט מציג את הקבצים שהועתקו + סטטיסטיקה. הסימון / אחרי המקור משמעותית — "תוכן התיקייה" לעומת "התיקייה עצמה".

🤖 **עצרו וחשבו:** דגלים: `-a` = archive (מחשור, timestamps, permissions)
`-v` = verbose, `-h` = human-readable sizes, `--delete` = מחק ב-dest קבצים שלא במקור (כדי לשקף בדיוק את המקור).

שלב 11 — שנו קובץ והריצו שוב

3

```
echo "ADDED: bonus clause" >> ~/business-data/contract.txt
sudo rsync -avh --delete ~/business-data/ /var/backups/rsync-local/
```

CLI

✓ **תוצאה צפויה:** הפעם הפלט קצר משמעותית — רק `contract.txt` מועתק.
 zé המהירות של rsync — מעביר רק delta.

חלק ד' — SSH Key Authentication (10 דקות)

לפני rsync מרוחק — צריך SSH ללא סיסמה. אחרת cron לא יוכל להריץ אוטומטית.

שלב 12 — צרו SSH key על linux01

1

```
ssh-keygen -t ed25519 -C "linux01@cyberlab" -f ~/.ssh/id_ed25519 -N ""
```

✓ **תוצאה צפויה:** 2 קבצים נוצרו: `~/.ssh/id_ed25519` (private) ו-`~/.ssh/id_ed25519.pub` (public).

⚠ **שימו לב:** `-N ""` אומר ללא passphrase. בארגון אמיתי = passphrase + ssh-agent. במעבדה = פשוט יותר.

שלב 13 — העתיקו את ה-public key ל-linux02

2

```
ssh-copy-id labadmin@192.168.200.20
```

בפעם הראשונה תופיע אזהרה "*The authenticity of host... can't be established*" → `yes`. אז יבקש סיסמה — הזינו את הסיסמה של labadmin על linux02.

✓ **תוצאה צפויה:** "*Number of key(s) added: 1*". בדקו:


```
ssh labadmin@192.168.200.20 "hostname"
```

CLI

SSH key . `cyberlab-linux02` **תוצאה צפויה:** ללא בקשת סיסמה — מקבלים authentication עובד. ✓

חלק ה' — rsync מרוחק על SSH (5 דקות)

שלב 14 — צרו destination ב-linux02

1

```
ssh labadmin@192.168.200.20 "sudo mkdir -p /var/backups/from-linux01 &&
```

CLI

תוצאה צפויה: תיקייה נוצרת על linux02 בבעלות labadmin (כדי שתוכלו לכתוב אליה דרך SSH key). ✓

שלב 15 — Sync ל-linux02

2

```
rsync -avh --delete ~/business-data/ labadmin@192.168.200.20:/var/backups
```

CLI

תוצאה צפויה: הפלט מציג העברה דרך SSH. בדקו ב-linux02: ✓

```
ssh labadmin@192.168.200.20 "ls -lh /var/backups/from-linux01"
```

CLI

✓ **תוצאה צפויה:** 2 הקבצים שלכם נמצאים על המכונה השנייה. גיבוי מרוחק עובד.

חלק ו' — אוטומציה עם cron (10 דקות)

שלב 16 — צרו סקריפט גיבוי

1

```
nano ~/backup.sh
```

CLI

תוכן הקובץ:

```
#!/bin/bash
SOURCE=~/business-data/
DEST=labadmin@192.168.200.20:/var/backups/from-linux01/
LOG=~/.backup.log

echo "=== Backup started at $(date) ===" >> $LOG
rsync -avh --delete $SOURCE $DEST >> $LOG 2>&1
EXIT_CODE=$?

if [ $EXIT_CODE -eq 0 ]; then
    echo "=== Backup SUCCESS at $(date) ===" >> $LOG
else
    echo "=== Backup FAILED (exit $EXIT_CODE) at $(date) ===" >> $LOG
fi
echo "" >> $LOG
```

CLI

Save → Exit. הוסיפו הרשאת הרצה:

```
chmod +x ~/backup.sh
~/backup.sh
```

CLI

```
cat ~/backup.log
```

✓ **תוצאה צפויה:** הסקריפט רץ. ה-log מציג success + start + פירוט .rsync.

שלב 17 — הוסיפו ל-cron

2

```
crontab -e
```

CLI

אם זו פעם ראשונה — בחרו `nano` כעורך. הוסיפו בסוף:

```
# Daily backup at 2:00 AM
0 2 * * * /home/labadmin/backup.sh
```

CLI

Save → Exit. בדקו:

```
crontab -l
```

CLI

✓ **תוצאה צפויה:** השורה מופיעה.

🤔 **עצרו וחשבו:** פורמט cron:

command day-of-week month day-of-month hour min . כוכבית = "כל".

= "כל יום ב-2:00 בוקר". `0 2 * * *`

שלב 18 — בדקו עם תזמון לדקה הקרובה

3

כדי לראות שהאוטומציה באמת עובדת — תזמנו ל-2 דקות הבאות:

```
date # בדקו את השעה הנוכחית
crontab -e
```

CLI

שנו את התזמון ל-(שעה_בוכחית + 2 דקות). לדוגמה אם השעה 14:35 — שימו

37 14 * * *

שמרו ועקבו:

```
watch -n 5 'tail -5 ~/backup.log'
```

CLI

✓ **תוצאה צפויה:** כשתגיע השעה — תראו את הסקריפט מתחיל וב-log מופיע
start + success. Ctrl+C לעצירת ה-watch.

החזירו ל-2:00 בלילה אחרי הבדיקה.

חלק ד' — Drill שחזור (5 דקות)

שלב 19 — מחקו את הקבצים על linux01

1

```
rm -rf ~/business-data
ls ~/business-data 2>&1
# אמור להגיד "No such file"
```

CLI

שלב 20 — שחזרו מ-linux02

2

```
mkdir -p ~/business-data
rsync -avh labadmin@192.168.200.20:/var/backups/from-linux01/ ~/business-data
ls ~/business-data
cat ~/business-data/contract.txt
```

CLI

✓ **תוצאה צפויה:** 2 הקבצים חזרו. השחזור עובד.

🤔 **עצרו וחשבו:** שימו לב — לא השתמשנו ב- `--delete` בשחזור. למה? כי אם היו קבצים ב-source שלא ב-backup, היינו רוצים לשמור אותם. בגיבוי השוטף — כן רוצים מחיקה כדי לשקף את המצב.

אתגרי בונוס

🏆 **אתגר בונוס: בונוס 1 — Incremental עם rsync וhard links.** תומך ב- `--link-dest` ליצירת snapshots חכמים. כל גיבוי = תיקייה חדשה, אבל קבצים שלא השתנו = hardlinks (לא תופסים מקום נוסף). חפשו ב-"Google" rsync time machine backup ובנו סקריפט שיוצר תיקייה לכל יום.

🏆 **אתגר בונוס: בונוס 2 — בדיקת integrity עם sha256.** אחרי גיבוי, צרו hash של כל קובץ במקור וב-יעד. השוו. אם hash שונה — הגיבוי פגום.

```
find ~/business-data -type f -exec sha256sum {} \;
```

🏆 **אתגר בונוס: בונוס 3 — שלחו התראה במקרה כישלון.** בסקריפט, אם `$EXIT_CODE` אינו 0 — שלחו אימייל. אם אין SMTP — כתבו ל-syslog: `logger -t backup "Backup failed!"`.

🏆 **אתגר בונוס: בונוס 4 — gzip + age cleanup.** שלבו rsync + tar. כל יום: tar.gz של היום, רק 7 ימים אחרונים נשמרים. — `find /var/backups -mtime +7 -delete` מחקו ישנים מ-7 ימים.

צ'ק-ליסט אימות

- ☐ 2 VMs של Ubuntu Server רצות (linux01 + linux02)
- ☐ ping בין שתי המכונות עובד

- SSH key authentication עובד (ssh ללא סיסמה) ☐
- rsync מ-linux01 ל-linux02 עובד ☐
- backup.sh רץ ומכתיב ב-log ☐
- cron rule מתוזמן יומי 02:00 ☐
- Restore Drill עבד — מחקתי, שחזרתי, הקבצים חזרו זהים ☐

השוואה Linux vs Windows Backup

WINDOWS (SERVER BACKUP)	LINUX (RSYNC + CRON)	היבט
חיום, חלק מ-OS	חיום, חלק מ-OS	Cost
VHDX images + metadata	קבצים גולמיים על FS	Format
VSS-based block-level	timestamps דרך delta	Incremental
Recovery Wizard	cp או rsync חזרה	Restore single file
מובנה ב-WSB	צריך כלי נוסף (Clonezilla)	Bare Metal
SMB / iSCSI	SSH (כל פלטפורמה)	Networking
Task Scheduler + PowerShell	cron + bash script	Automation

בסוף המעבדה

1. קחו snapshot על שתי ה-VMs: `Post-Lab-04-LinuxBackup`
2. `sudo poweroff` בשתיהן

[DR Documentation :מעבדה 05 ← הבא](#)

[Backup VSS :הקודם →](#)

29/04/2026 · מעבדה 04 · ניהול מערכות ושירותים · מעבדה 04 — מודול 5 · CyberLabs ©